

SIMD Clustering Algorithms with EVE in C++

Internship Report

Rémi Millerat–Gallot

M1 MPRI — Université Paris-Saclay

Host laboratory: LISN — Laboratoire Interdisciplinaire des Sciences du Numérique
Team: ParSys
Laboratory supervisor: Joël Falcou
Academic supervisor: Burkhard Wolff
Internship period: 20 April 2026 – 17 July 2026

Abstract

This report presents my internship work at LISN in the ParSys team on SIMD implementations of clustering algorithms in C++20 using the EVE library. The codebase covers Lloyd’s K-Means, greedy K-Means++ initialization, Gaussian Mixture Models trained by Expectation-Maximization, and a dense Euclidean HDBSCAN pipeline.

The main implementation effort compared these across different dimensional regimes; they were integrated into a validation and benchmarking pipeline against scikit-learn Python references.

The observed performance gains depend on the fit between the vectorization strategy, memory layout, and algorithmic dataflow. Against the scikit-learn reference, validated end-to-end timings ranged from near parity to approximately $100\times$ faster. Speedups of $5\text{--}15\times$ were common in representative, non-extreme configurations, although the precise gain varied substantially with the algorithm and the values of N , D , and K .

Acknowledgements

I would first like to thank Joël Falcou, without whom this internship wouldn't have gone that far. His technical ideas, guidance, and comments were central to the direction of the project, while still leaving me the freedom to explore the implementation choices and difficulties by myself.

I would also like to thank Alexis Aune, Joël Falcou's PhD student and my office-mate during the internship, and the rest of the ParSys team for their kindness and for contributing to a pleasant working environment at LISN.

Use of Generative AI Tools

During this internship, I made extensive use of generative AI tools as programming and writing assistants. They were used throughout the project to help produce code, modify and debug implementations, write documentation, and structure this report. In practice, essentially all the code in the project was produced with substantial AI assistance, then reviewed, tested, selected, and integrated by me. This use of generative AI should therefore be understood as part of the working methodology of the internship, rather than as a minor editorial aid. The final responsibility for the submitted repository, experiments, documentation, and results remains mine; in particular, the scientific and technical work of understanding the clustering algorithms, choosing implementation directions, validating behavior, and interpreting benchmark results remains my responsibility.

1 Internship context and project overview

1.1 Host laboratory and scientific environment

This internship took place at LISN, the Laboratoire Interdisciplinaire des Sciences du Numérique, within the ParSys team, as part of the first year of the MPRI master's program at Université Paris-Saclay. The internship ran from 20 April 2026 to 19 July 2026, under the supervision of Joël Falcou at LISN and Burkhard Wolff at Université Paris-Saclay.

LISN provided the broader academic environment of the internship. The ParSys team, short for *Systèmes Parallèles* and part of the “Algorithmes, Apprentissage et Calcul” research department, provided the more specific scientific setting. Its work on parallel systems, high-performance computing, and efficient implementation of algorithms on modern architectures was directly related to the subject of the internship. The project was therefore situated at the intersection of unsupervised learning and high-performance implementation: the algorithms considered come from data analysis and machine learning, while their implementation requires attention to memory layout, vectorization, register pressure, numerical stability, and the cost of data movement.

1.2 Internship organization and working mode

The work was mostly carried out independently, with comments and feedback from the laboratory supervisor. There was no detailed initial specification or fixed list of expected deliverables. This organization had an important influence on the nature of the work. A significant part of the internship consisted not only in writing optimized code, but also in defining the experimental perimeter itself: which algorithms were reasonable to study, which computations should be isolated as kernels, how Python and C++ implementations should be compared, what should be measured, and under which conditions a performance result could be considered meaningful.

At the beginning of the internship, I had limited prior experience with C++ and SIMD programming. Apart from approximately twenty hours in Joël Falcou's C++ course, I had not previously developed a large C++ project or studied SIMD-oriented algorithms in depth. I had already encountered clustering algorithms in a previous course, but not from the point of view

of low-level implementation. The internship was therefore also a learning process, in which the programming tools, the performance model, and the validation methodology had to be developed alongside the code itself.

1.3 Project overview and problem statement

The general topic of the internship was the implementation of performant clustering algorithms in C++20 using EVE. The project was not intended to produce code ready for immediate integration into EVE or to build a complete replacement for existing Python libraries such as scikit-learn [1]. The project instead became a benchmark and validation codebase for clustering-related algorithms and numerical kernels. Its central question was how data layout and kernel organization affect performance as the number of features, samples, and clusters varies.

The code covers Lloyd’s K-Means, greedy K-Means++ initialization, Gaussian Mixture Model estimation by Expectation-Maximization, and a dense Euclidean HDBSCAN pipeline. The code was designed to study how these computations behave under different dimensional regimes. It includes both static-D and dynamic-D variants; their respective data layouts and traversal strategies are described in the technical background below.

You can find the project’s repository [on github](#).

2 Theoretical and technical background

2.1 Clustering algorithms

Clustering is an unsupervised learning task in which a set of observations $x_1, \dots, x_N \in \mathbb{R}^D$ is organized into groups without using external labels. In this project, clustering was mainly used as a source of numerical kernels with different computational structures: repeated Euclidean distance computations, reductions over points or clusters, probabilistic density evaluations, and nearest-neighbour or graph-based operations.

Lloyd’s K-Means algorithm seeks K centroids $c_1, \dots, c_K \in \mathbb{R}^D$ minimizing the within-cluster sum of squared distances

$$\sum_{i=1}^N \min_{1 \leq k \leq K} \|x_i - c_k\|_2^2.$$

The standard iterative algorithm alternates between assigning each point to its nearest centroid and recomputing each centroid as the mean of its assigned points [2]. From an implementation point of view, the assignment step is the most relevant part for SIMD acceleration, since it requires many independent distance computations between points and centroids.

K-Means++ modifies only the initialization step. In its original form, the first center is chosen uniformly at random, and each subsequent center is sampled with probability proportional to its squared distance from the nearest previously selected center [3]. A greedy variant [4] draws ℓ such candidates at each subsequent step and retains the one minimizing the updated potential

$$\Phi(C) = \sum_{i=1}^N \min_{c \in C} \|x_i - c\|_2^2;$$

setting $\ell = 1$ recovers vanilla K-Means++. It increases initialization work by evaluating several candidates per selected center, but can yield a lower initial potential.

Gaussian Mixture Models use a probabilistic formulation. Instead of assigning each point to a single cluster center, the data is modeled as a mixture of K Gaussian distributions with weights π_k , means μ_k , and covariance matrices Σ_k . Parameters are typically estimated by the Expectation-Maximization algorithm [5]. The E-step computes responsibilities, that is, soft assignments of points to mixture components, while the M-step updates the parameters from these responsibilities. For this internship, GMMs were interesting because they reuse distance-like

computations but add more complex numerical issues: normalization, exponentials, covariance handling, and stronger sensitivity to floating-point differences.

HDBSCAN is a density-based clustering method related to DBSCAN [6], but it avoids selecting a single global density threshold. It builds a hierarchy using mutual-reachability distances derived from core distances, then extracts stable clusters from this hierarchy [7, 8]. Compared with K-Means and GMMs, HDBSCAN has a less regular computational structure because it involves nearest-neighbour information and graph construction.

2.2 SIMD programming and EVE

Single Instruction Multiple Data (SIMD) programming is a form of data-level parallelism in which one instruction is applied simultaneously to several scalar values. Instead of executing the same arithmetic operation repeatedly on individual values, a processor applies it to a vector register containing several values. For example, a SIMD addition instruction can add several pairs of floating-point numbers in one machine instruction.

SIMD is different from task parallelism. It does not mean that several independent threads execute different pieces of code. Rather, one instruction stream is kept, but each instruction operates on multiple data lanes. This makes SIMD particularly suitable for regular data-parallel computations.

A SIMD register can be seen abstractly as a small fixed-size array:

$$[x_0, x_1, x_2, \dots, x_{w-1}],$$

where w is the number of scalar lanes. The value of w depends on both the scalar type and the hardware vector width. For instance, a 256-bit SIMD register can hold eight 32-bit floats, four 64-bit doubles, or eight 32-bit integers. This is one reason why the choice between `float` and `double` has a direct impact on SIMD throughput: using single precision doubles the number of values processed per vector instruction [9].

SIMD programming occupies an intermediate position between scalar CPU programming and GPU programming. GPUs provide massive parallelism and are effective for large, regular workloads with high arithmetic intensity, but they introduce constraints such as device-memory transfers, kernel-launch overhead, a less direct integration with ordinary CPU code, and often higher power consumption. SIMD, by contrast, exploits parallelism already present inside the CPU. It targets a smaller scale of parallelism than GPUs, with only a limited number of lanes per core, but it has lower overhead, works naturally on CPU-resident data, and can be combined with cache-aware layouts, loop unrolling, compiler optimizations, and multithreading. In this sense, SIMD is often a first level of explicit performance engineering for numerical code on CPUs, rather than a replacement for GPU programming.

At the hardware level, SIMD support is exposed through architecture-specific instruction sets. On x86 processors, common SIMD extensions include SSE, AVX, AVX2, and AVX-512, while on ARM processors similar roles are played by NEON and SVE. Programmers can access these instructions through CPU intrinsics, which are compiler-provided functions that map closely to individual machine instructions and allow explicit use of vector operations, shuffles, masks, comparisons, and reductions. However, intrinsics are verbose and difficult to make portable, because instruction sets differ in register width, supported operations, masking capabilities, and sometimes programming model. For example, AVX2 intrinsic code cannot in general be reused unchanged on an ARM processor, and AVX-512 code may need to be rewritten or specialized for machines that only support AVX2.

Modern C++ SIMD libraries provide a higher-level abstraction over architecture-specific intrinsics. EVE is one such library [10]: it provides C++ types and functions for expressing SIMD operations portably, while still allowing the compiler to generate efficient machine-specific instructions. Instead of writing separate intrinsic code for each instruction set, the programmer writes vector code using EVE abstractions, which are then mapped to the appropriate SIMD

backend when possible. This does not remove the need to reason about SIMD: memory access patterns, vector width, masks, reductions, and data layout remain important. However, EVE separates the algorithmic expression of vector operations from many low-level details, making it possible to write SIMD-oriented code that stays closer to ordinary C++ while still exposing explicit vectorization opportunities.

In clustering algorithms, SIMD is naturally relevant because many operations are repeated over large collections of points. Distance computations, assignment steps, likelihood evaluations, and reductions all contain regular numerical kernels. The main difficulty is not only to identify these arithmetic operations, but also to organize the data so that values needed together are contiguous or easily loadable into SIMD registers. For this reason, SIMD programming is closely linked to the choice of data layout, which is discussed in the next subsection.

2.3 Data layout: AoS, SoA, static-D, and dynamic-D

A common preliminary question in SIMD implementations of dense numerical algorithms is how vector lanes should be interpreted. For a dataset represented as a dense array of shape (N, D) , with N samples and ambient dimension D , SIMD can in principle be applied along different axes of the data. One possibility is to vectorize across the dimensions of a single sample. In that case, an array-of-structures layout, or ordinary row-major layout, is natural: the coordinates $x_{i,0}, x_{i,1}, \dots, x_{i,D-1}$ of one sample are contiguous, so one SIMD load can contain several coordinates of the same point. This organization is often appropriate for kernels that behave like dot products or reductions over the coordinates of one object. Another possibility is to vectorize across samples while keeping the dimension fixed. Then a structure-of-arrays layout is more natural [11]: for a fixed coordinate d , the values $x_{0,d}, x_{1,d}, \dots, x_{W-1,d}$ of several consecutive samples are contiguous and can be loaded into one SIMD register. This second convention, also used by Otsuka and Fukushima for AVX/AVX2 implementations of K-Means and K-Means++ [12], is particularly useful in clustering kernels where the same centroid coordinate, threshold, or model parameter is applied to many samples independently. The static-dimensional and dynamic-dimensional layouts described below follow this lanes-as-samples convention, while differing in how the dimension axis itself is represented and traversed.

When the dimension is small and fixed, one possible organization is to represent a block of samples as a fixed collection of SIMD vectors, with one vector per coordinate. For instance, in dimension three, loading one SIMD block of samples gives

$$([x_0, x_1, \dots], [y_0, y_1, \dots], [z_0, z_1, \dots]).$$

The coordinates are then not traversed as an ordinary runtime loop: the computation can be expressed directly over this fixed collection of vectors. For example, a squared Euclidean distance to a point c can be written as

$$(x - c_x)^2 + (y - c_y)^2 + (z - c_z)^2,$$

where each term is itself a SIMD operation over several samples. This exposes the whole coordinate expression to the compiler and makes low-dimensional kernels compact and specialized.

The cost of this organization is that all coordinates of the current block are live at the same time. In dimension D , this already requires D SIMD registers before considering accumulators, masks, constants, candidate parameters, or other temporary values. As D grows, the issue is therefore not the number of arithmetic operations alone, but the size of the live working set. If too many values have to be kept simultaneously, the compiler may run out of available registers and spill intermediate data to memory [13].

An alternative organization is to store the data in dimension-major form and to traverse the coordinate axis progressively, for example

$$\text{samples}[d][n : n + W].$$

At a given point, the computation only needs the currently loaded coordinate vector, or a small block of such vectors, together with the intermediate quantities that are deliberately kept live. This does not imply scalar code: the coordinate traversal may still be specialized, unrolled, tiled, or blocked. The main design question is how much of the coordinate structure should be materialized at once. A fully materialized organization exposes the complete coordinate expression to the compiler, which is often attractive in low dimension. A streamed or blocked organization instead controls the live working set by processing coordinates in smaller pieces; even when the dimension is known, this can be preferable for larger dimensions or for kernels that require many accumulators, parameters, sample blocks, or candidate objects to remain live simultaneously.

In all cases, the memory layout should match the chosen interpretation of SIMD lanes. When lanes represent samples, contiguous memory should provide the same coordinate for consecutive samples. When lanes represent coordinates, contiguous memory should provide consecutive coordinates of the same sample. The static-dimensional and dynamic-dimensional layouts introduced later in the implementation chapter are two concrete points in this broader design space, not universal categories.

3 Implementation and benchmark methodology

The implementation work was organized around a common benchmark architecture rather than around a single isolated kernel. Each benchmark case runs one configured C++ path and records algorithm-level metrics. This made it possible to benchmark several clustering algorithms under comparable conditions while still allowing each algorithm to use a specialized SIMD layout internally.

3.1 Implemented benchmark space

The repository contains benchmark cases for the four clustering algorithms considered, with multiple implementations for each, and layout conversion. Layout conversion is included because the optimized kernels do not operate directly on arbitrary row-major input arrays, but need data in the corresponding SIMD-oriented layouts described above.

Case	implemented variants
SoA conversion	static-D, dynamic-D
Lloyd K-Means	static-D, dynamic-D, auto-dispatch
K-Means++	static-D, dynamic-D
GMM	static-D, dynamic-D; spherical, diagonal, full covariance types
HDBSCAN	static-D: distance, MST, linkage, selection, full pipeline

Table 1: Implemented benchmark cases.

The Lloyd implementation uses a shared driver for the algorithmic structure: assigning samples, updating centers, repairing empty clusters, and recording metrics. The SIMD-specific work is concentrated in the assignment backends. The static-D and dynamic-D backends correspond to the previously explained strategies. The dynamic-D backend is itself split into two different assignments backends, with different tiling strategies.

The GMM implementation follows the same lanes-as-samples convention as Lloyd, but its E-step requires scores for all components before responsibilities can be normalized. It therefore uses block-local score storage and immediately accumulates the sufficient statistics needed by the M-step. The covariance model is represented as a policy, allowing the same EM structure to support spherical, diagonal, and full covariance cases. The full-covariance path is the most demanding because its scoring and accumulation structure scales with D^2 per component.

The HDBSCAN implementation is a dense Euclidean staged pipeline in double precision. The full path computes the distance matrix, overwrites it with mutual-reachability distances, constructs a minimum spanning tree, converts it into a single-linkage hierarchy, and applies cluster selection. The same pipeline can also be run stage by stage, which was important for validation because the later HDBSCAN stages are less SIMD-regular and are more sensitive to data contracts and tie-breaking behavior than to arithmetic throughput alone. The optimized C/C++ implementation and report of Frey et al. served as a source of candidate optimizations, several of which were reimplemented with EVE and evaluated for our HDBSCAN path [14].

3.2 Benchmark and validation methodology

The benchmark methodology is designed to compare equivalent computations rather than merely comparable-looking timings. The C++ implementations and the Python reference paths therefore share materialized input artifacts whenever possible. Synthetic datasets are generated once, stored in a contiguous binary format, and reused by both sides. Lloyd and GMM benchmarks also reuse initialization artifacts: Lloyd starts from shared K-Means++ centers, while GMM derives its initial weights, means, and covariance or precision parameters from the same centers. This keeps the measured Lloyd and GMM runs deterministic for a fixed configuration and avoids attributing initialization differences to the implementation’s performance.

Setup work that is not part of the algorithm being evaluated (including input preparation and output handling) is excluded from the measured region; only the fitting computation is timed. Native thread pools are restricted to one thread so that the results focus on single-thread SIMD effects rather than thread-level parallelism.

C++ timings are collected with [ankerl::nanobench](#), and Python timings with [pyperf](#). Postprocessing discards warmup and calibration values, groups repeated runs, and reports descriptive statistics for total time. For iterative algorithms, time per iteration is obtained by dividing the measured time by the recorded iteration count. Speedup is defined as

$$\text{speedup} = \frac{\text{Python reference time}}{\text{C++ time}},$$

so values greater than one mean that the C++ implementation is faster under the benchmark contract.

Validation is used before interpreting speedups. Repeated C++ timing processes must agree on recorded algorithm metrics before their timing files are merged. C++ outputs are then compared with Python reference outputs when the comparison is well-defined. Lloyd is checked through final inertia and iteration count. GMM is checked through EM iteration count, lower bound, weights, means, and covariance quantities. HDBSCAN is checked both through selected intermediate stages and through the full dense pipeline. K-Means++ is treated separately as a timing benchmark for the initialization procedure, because the measured Python and C++ paths do not use an identical sampled sequence. Configurations that do not pass the relevant parity checks may still be useful for debugging, but they are not presented as clean reference-equivalent speedups.

The detailed artifact formats, HDBSCAN stage conventions, tolerance rules, and parity scripts are documented in `documentation/benchmark_methodology.md`, and in `documentation/scikit_parity_and_validation.md`.

3.3 Diagnostics

The same case-adaptor mechanism can run selected configurations normally, through nanobench, through Valgrind’s Callgrind with cache simulation enabled [15], or through a spill-detection path. The spill detector compiles selected cases to assembly and scans for stack spill and reload patterns, but it is only a heuristic. The full benchmark pipeline also allows these tools to run after each performance measurements, so we can have a full overview of cache and register behavior.

4 Results and analysis

The complete tables, plots, raw timing summaries, parity logs, portability measurements, analysis and interpretations are provided in `/deliverables/latest_results/`, and `documentation/performance_results.md`.

4.1 Performance and speedup scaling by algorithm

The measured gains vary substantially by algorithm because the C++ paths do not benefit from the same optimizations or face the same bottlenecks. The following table summarizes the representative end-to-end results, measured on a Zen 5 CPU (AVX512) in WSL 2, with `g++-15`.

Algorithm	Representative speedup	Caveat
Lloyd	5–13×	The only main optimization not present in the reference is delta updates for the M step, which only provides around a 10 % boost
GMM EM	~15–70×	A substantial part of the speedup comes from fusing the E step with the accumulation of M-step statistics.
HDBSCAN	~3–7×	I didn't implement the dynamic-D variant <code>*yet*</code> , so speedups aren't as good as they could be.
K-Means++	~10–35×	No formal parity check is currently available, so these results should not be interpreted too strongly.

Table 2: Summary of performance and speedup scaling by algorithm.

Overall, the dynamic-D variants outperform the static-D variants in almost every tested configuration, including the low-dimensional regions in which the static-D representation should be most favorable. This is likely because the dynamic-D kernels are more aggressively optimized: they tile across samples and centroids, components, or candidates; reuse loaded data and coefficients; expose several independent accumulator chains; and explicitly control register pressure. By contrast, the static-D variants were intentionally designed as simpler, idiomatic EVE baselines, relying heavily on EVE algorithms and tuple traversal rather than explicit blocking across samples and clusters. They could be improved through multi-sample and multi-cluster tiling, multiple accumulators, register-aware blocking, and specialized low-dimensional microkernels. That was not the main purpose of these simple variants, but it is a natural direction for future work: a blocked static-D implementation might combine these optimizations with whole-dimension unrolling and better reuse of an in-register sample tuple, potentially allowing it to outperform the dynamic-D variants in the low-dimensional regimes where that representation is most advantageous.

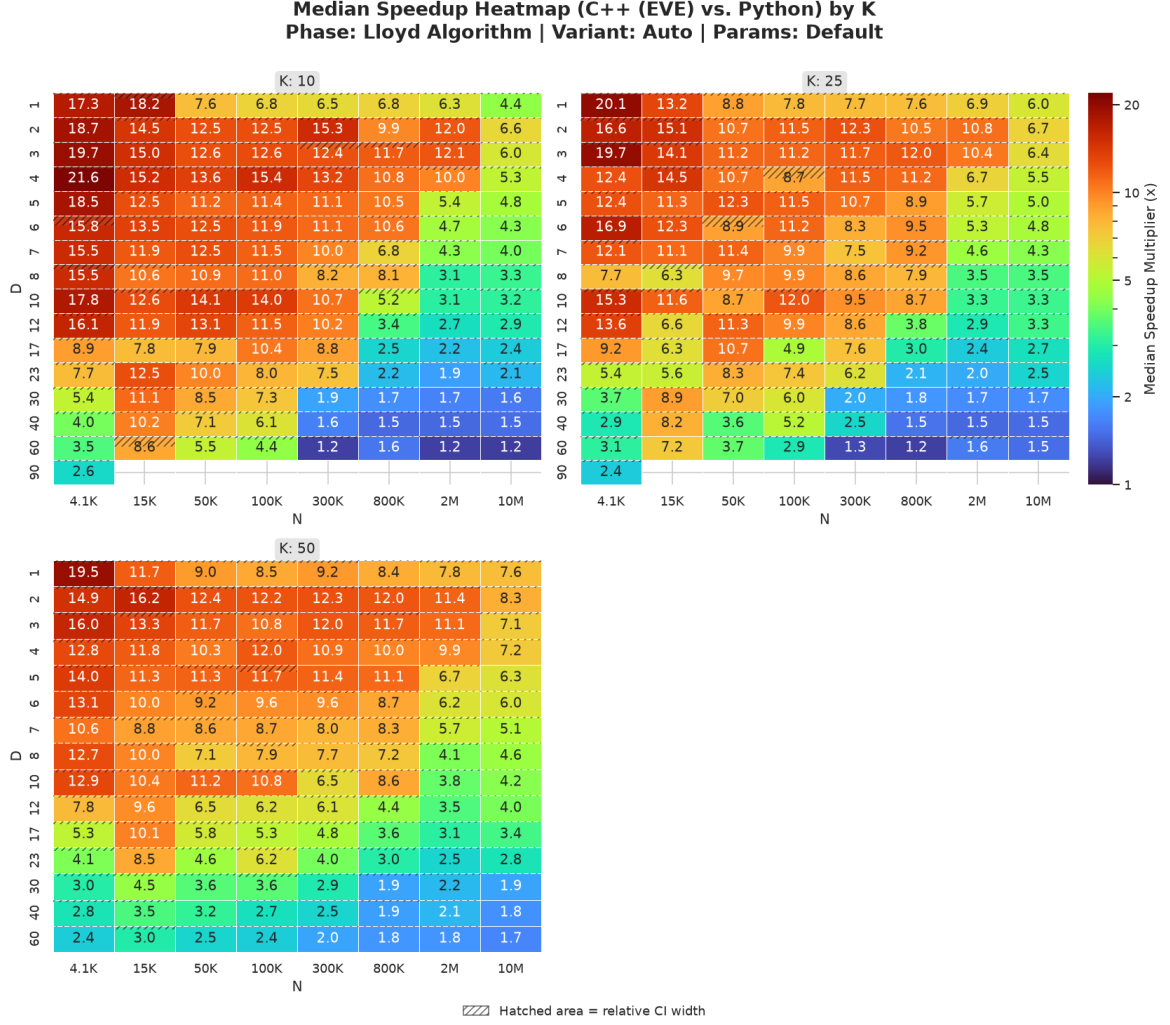


Figure 1: Speedup of the ‘automatic’ Lloyd variant over the scikit-learn reference on the AMD Zen 5 machine. The speedup depends on the number of samples N , dimensions D , and clusters K .

4.2 Portability across compilers and machines

I ran the benchmarks on three different machines:

Table 3: Speedup difference against the Zen 3 CPU baseline. Not performance difference: if both scikit and C++ scaled the same, the speedup difference stays 1.

Machine	Lloyd K-means	K-means++	GMM EM	HDBSCAN
Zen 5 (AVX-512, WSL 2)	1.5	1.2	2.3	0.9
Zen 3 (AVX2, WSL 2)	1.0	1.0	1.0	1.0
Rocket Lake (native Linux)	1.6	1.0	0.6	(not measured)

I also ran the benchmarks with clang++ 18. At first performance was much lower, but increasing the `-inline-threshold` value did bring it up to g++’s level, except for GMM EM where it’s only 0.85× as fast.

5 Limitations and future work

- Turn the current codebase into a complete clustering library, with a stable API, estimator classes, and broad input validation.
- Reproduce a wider range of scikit-learn behavior and support additional use cases, including non-dense inputs.
- Implement more clustering algorithms, with parity validation for each implementation.
- Investigate and improve performance stability across compilers and architectures, as well as performance scaling across different hardware capabilities.
- Improve automatic dispatch and extend it to all supported algorithms.
- Add thread-level parallelism, both as a separate optimization and in combination with SIMD.
- Benchmark against additional reference implementations and on a broader range of real-world datasets.
- Maintain multiple versions of the implementations to isolate the effects of individual optimizations and, in particular, to measure the contribution of SIMD alone.

References

- [1] F. Pedregosa et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. <https://jmlr.csail.mit.edu/papers/v12/pedregosa11a.html>.
- [2] S. P. Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982. <https://doi.org/10.1109/TIT.1982.1056489>.
- [3] D. Arthur and S. Vassilvitskii. k-means++: The advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1027–1035. SIAM, 2007. <https://theory.stanford.edu/~sergei/papers/kMeansPP-soda.pdf>.
- [4] C. Grunau, A. A. Özüdoğru, V. Rozhoň, and J. Tětek. A nearly tight analysis of greedy k-means++. In *Proceedings of the 2023 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1012–1070. SIAM, 2023. <https://doi.org/10.1137/1.9781611977554.ch39>.
- [5] A. P. Dempster, N. M. Laird, and D. B. Rubin. Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society: Series B (Methodological)*, 39(1):1–38, 1977. <https://doi.org/10.1111/j.2517-6161.1977.tb01600.x>.
- [6] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, pages 226–231. AAAI Press, 1996. <https://doi.org/10.5555/3001460.3001507>.
- [7] R. J. G. B. Campello, D. Moulavi, and J. Sander. Density-based clustering based on hierarchical density estimates. In *Advances in Knowledge Discovery and Data Mining*, volume 7819 of *Lecture Notes in Computer Science*, pages 160–172. Springer, 2013. https://doi.org/10.1007/978-3-642-37456-2_14.
- [8] L. McInnes, J. Healy, and S. Astels. hdbscan: Hierarchical density based clustering. *Journal of Open Source Software*, 2(11):205, 2017. <https://doi.org/10.21105/joss.00205>.

- [9] Intel Corporation. Intel Intrinsic Guide, version 3.6.9. 2024. <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html>.
- [10] J. Pénuchot, J. Falcou, and A. Khabou. Modern generative programming for optimizing small matrix-vector multiplication. In *2018 International Conference on High Performance Computing & Simulation (HPCS)*, pages 508–514, 2018. <https://doi.org/10.1109/HPCS.2018.00086>.
- [11] S. Jubertie, I. Masliah, and J. Falcou. Data layout and SIMD abstraction layers: decoupling interfaces from implementations. In *2018 International Conference on High Performance Computing & Simulation (HPCS)*, pages 531–538, 2018. <https://doi.org/10.1109/HPCS.2018.00089>.
- [12] T. Otsuka and N. Fukushima. Vectorized implementation of K-means. In *International Workshop on Advanced Imaging Technology (IWAIT) 2021*, Proceedings of SPIE, volume 11766, article 1176631, 2021. <https://doi.org/10.1117/12.2590842>.
- [13] Intel Corporation. *Intel 64 and IA-32 Architectures Optimization Reference Manual*, Volume 1, Order No. 248966, revision 049. <https://cdrdv2-public.intel.com/814198/248966-Optimization-Reference-Manual-V1-049.pdf>.
- [14] M. Frey, B. Hubmann, M. Tschechne, and R. Worreby. Hierarchical density based clustering. Project report, Advanced Systems Lab, ETH Zurich, 2021. https://github.com/MartinTschechne/ASL-hdbscan/blob/master/33_report.pdf.
- [15] N. Nethercote and J. Seward. Valgrind: a framework for heavyweight dynamic binary instrumentation. In *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 89–100, 2007. <https://doi.org/10.1145/1250734.1250746>.